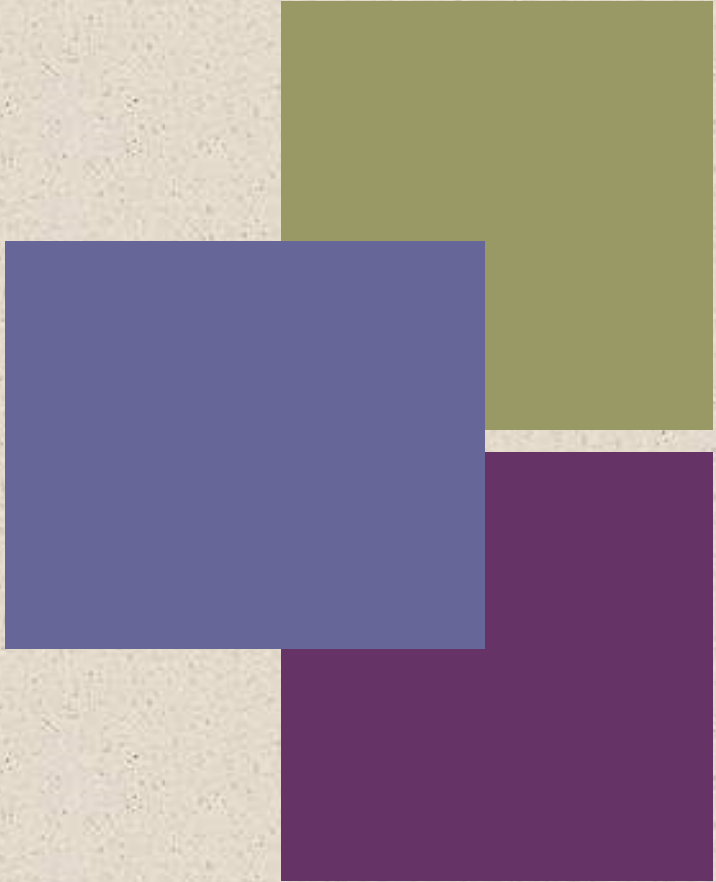




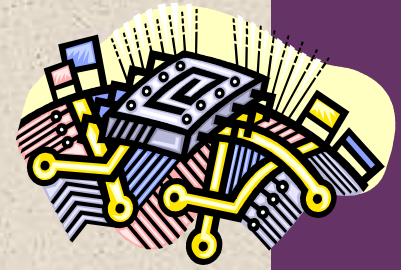
+ Chapter 14

Processor Structure and Function



Processor Organization

Processor Requirements:



- **Fetch instruction**
 - The processor reads an instruction from memory (register, cache, main memory)
- **Interpret instruction**
 - The instruction is decoded to determine what action is required
- **Fetch data**
 - The execution of an instruction may require reading data from memory or an I/O module
- **Process data**
 - The execution of an instruction may require performing some arithmetic or logical operation on data
- **Write data**
 - The results of an execution may require writing data to memory or an I/O module
- In order to do these things the processor needs to store some data temporarily and therefore needs a small internal memory

Instruction Cycle

Includes the following stages:

Fetch

Read the next instruction from memory into the processor

Execute

Interpret the opcode and perform the indicated operation

Interrupt

If interrupts are enabled and an interrupt has occurred, save the current process state and service the interrupt

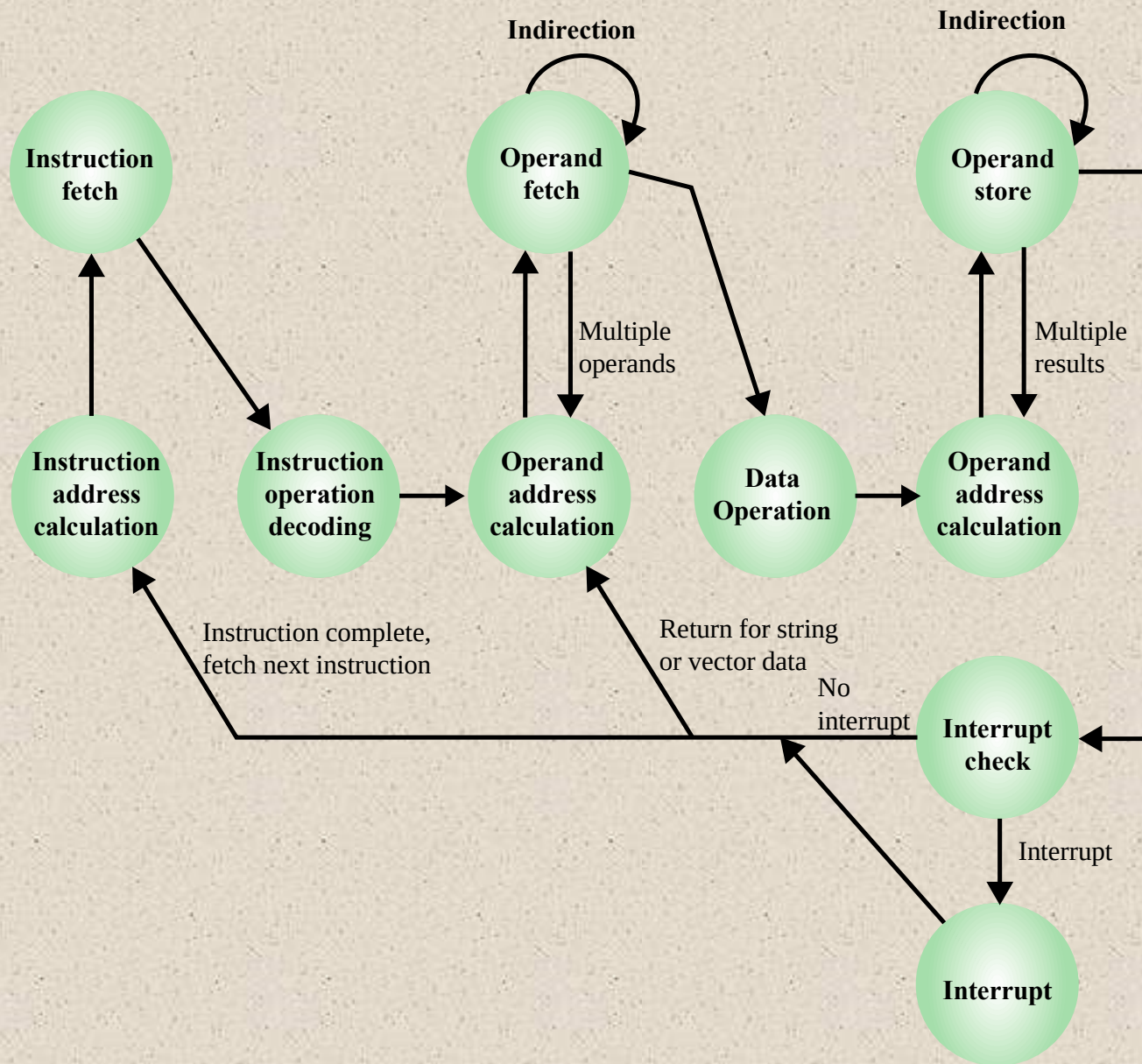
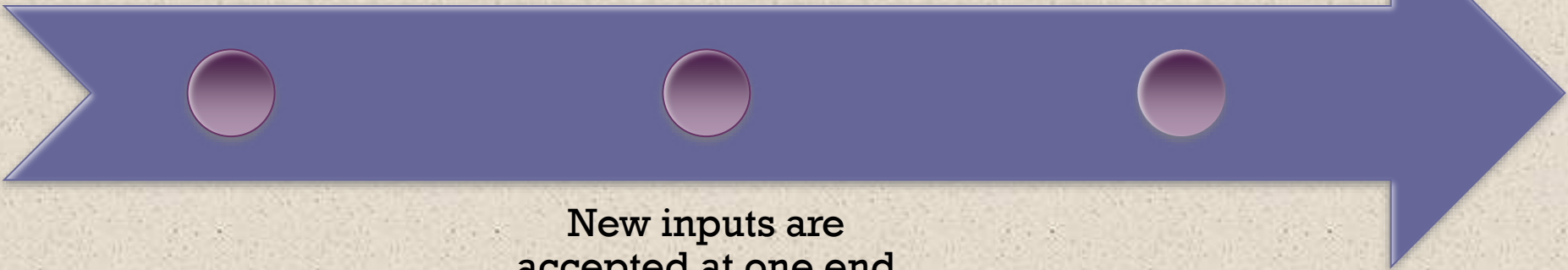


Figure 14.5 Instruction Cycle State Diagram

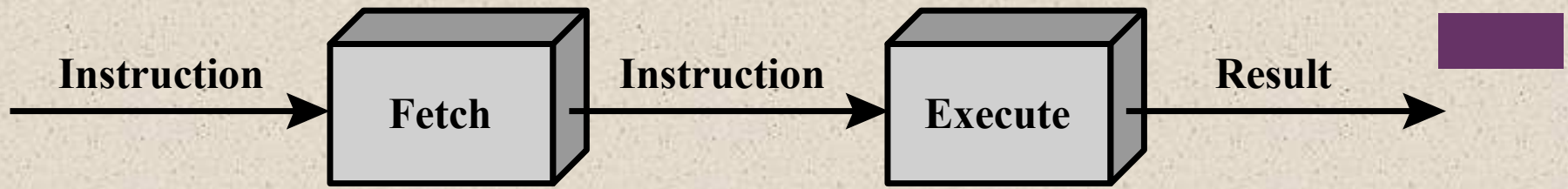
Pipelining Strategy

Similar to the use of
an assembly line in a
manufacturing plant

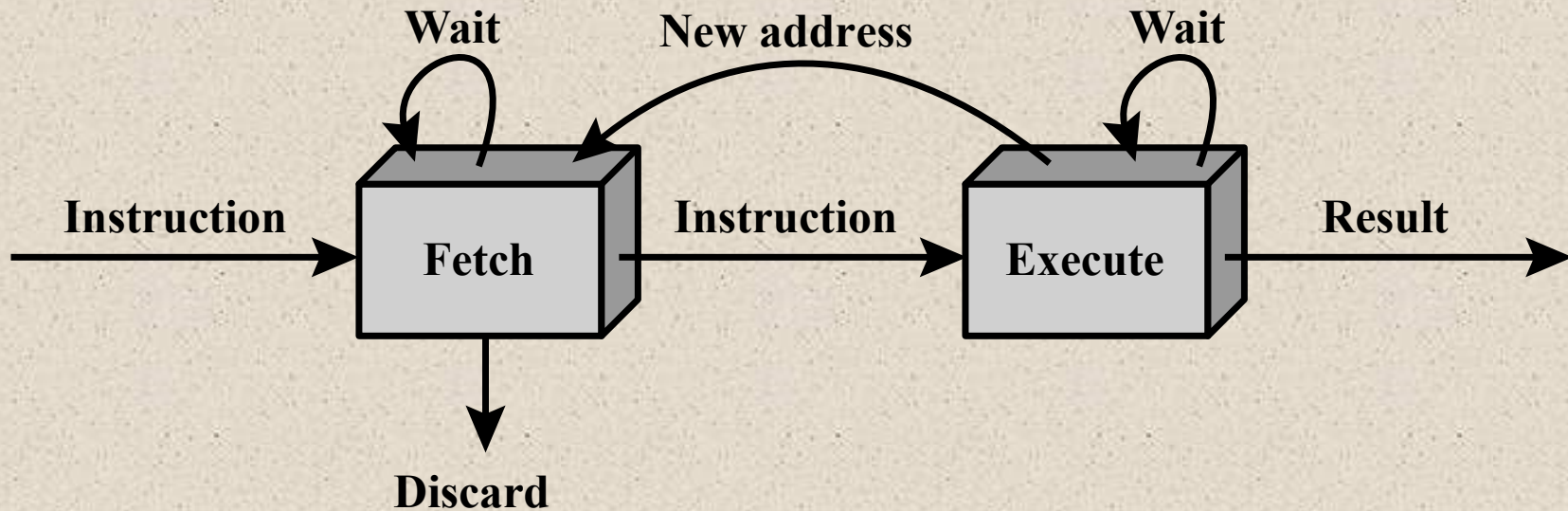
To apply this concept
to instruction
execution we must
recognize that an
instruction has a
number of stages



New inputs are
accepted at one end
before previously
accepted inputs
appear as outputs at
the other end



(a) Simplified view



(b) Expanded view

Figure 14.9 Two-Stage Instruction Pipeline

+ Additional Stages

- Fetch instruction (FI)
 - Read the next expected instruction into a buffer
- Decode instruction (DI)
 - Determine the opcode and the operand specifiers
- Calculate operands (CO)
 - Calculate the effective address of each source operand
 - This may involve displacement, register indirect, indirect, or other forms of address calculation
- Fetch operands (FO)
 - Fetch each operand from memory
 - Operands in registers need not be fetched
- Execute instruction (EI)
 - Perform the indicated operation and store the result, if any, in the specified destination operand location
- Write operand (WO)
 - Store the result in memory

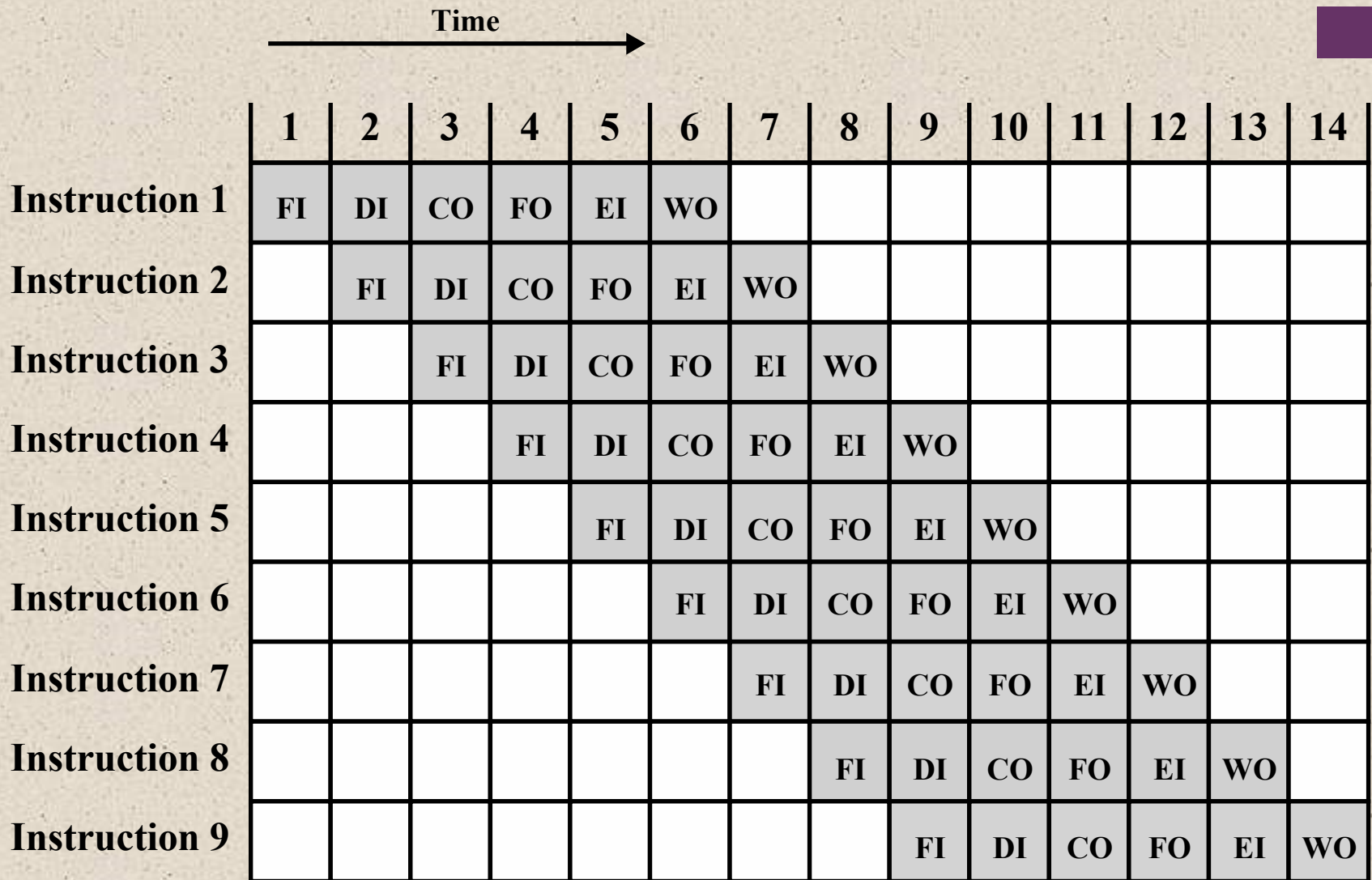


Figure 14.10 Timing Diagram for Instruction Pipeline Operation

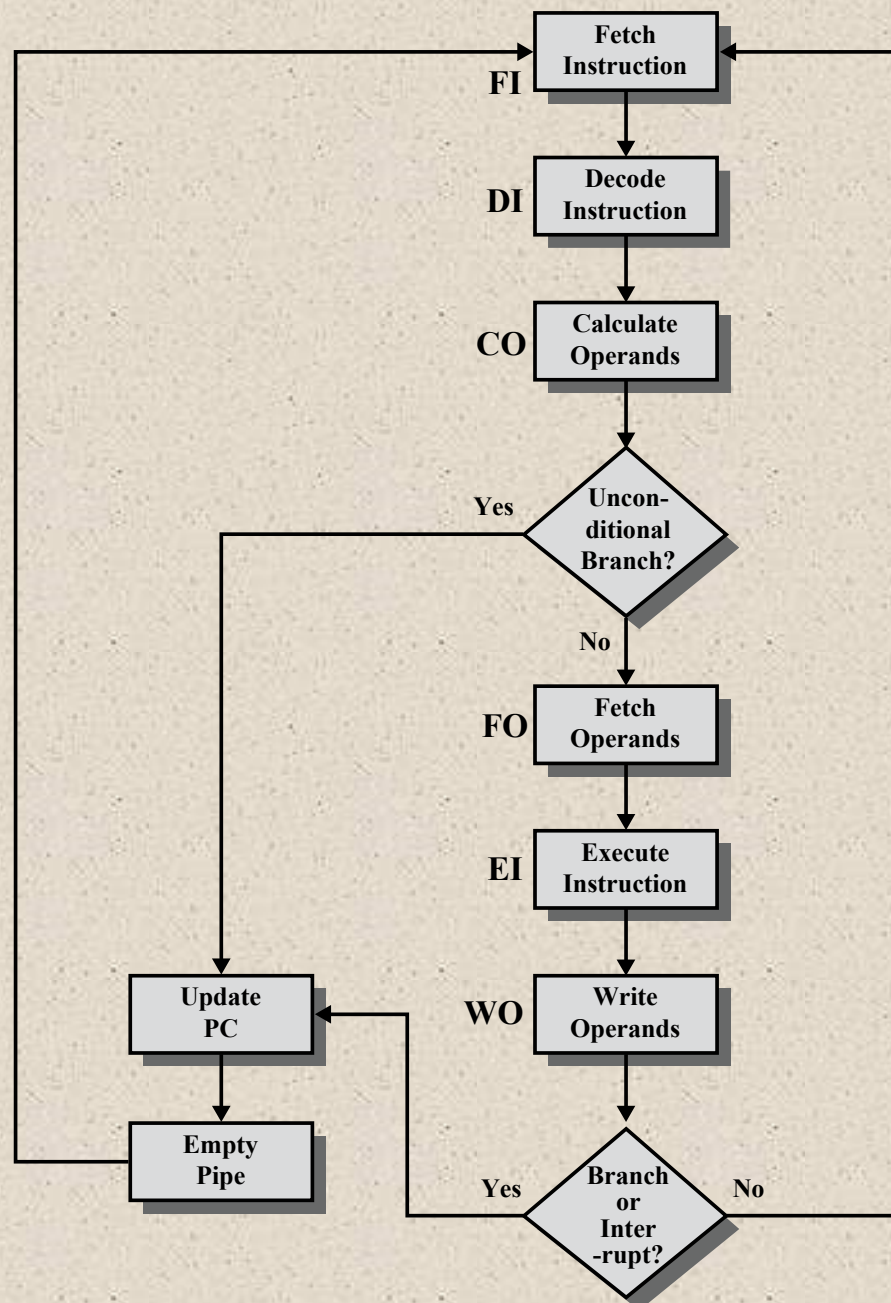


Figure 14.12 Six-Stage Instruction Pipeline

Time
↓

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I8	I7	I6	I5	I4	I3
9	I9	I8	I7	I6	I5	I4
10		I9	I8	I7	I6	I5
11			I9	I8	I7	I6
12				I9	I8	I7
13					I9	I8
14						I9

(a) No branches

	FI	DI	CO	FO	EI	WO
1	I1					
2	I2	I1				
3	I3	I2	I1			
4	I4	I3	I2	I1		
5	I5	I4	I3	I2	I1	
6	I6	I5	I4	I3	I2	I1
7	I7	I6	I5	I4	I3	I2
8	I15					I3
9	I16	I15				
10		I16	I15			
11			I16	I15		
12				I16	I15	
13					I16	I15
14						I16

(b) With conditional branch

Figure 14.13 An Alternative Pipeline Depiction

Pipeline Hazards

Occur when the pipeline, or some portion of the pipeline, must stall because conditions do not permit continued execution

There are three types of hazards:

- Resource
- Data
- Control

Also referred to as a *pipeline bubble*



		Clock cycle								
		1	2	3	4	5	6	7	8	9
Instrucion	I1	FI	DI	FO	EI	WO				
	I2		FI	DI	FO	EI	WO			
	I3			FI	DI	FO	EI	WO		
	I4				FI	DI	FO	EI	WO	

(a) Five-stage pipeline, ideal case

		Clock cycle								
		1	2	3	4	5	6	7	8	9
Instrucion	I1	FI	DI	FO	EI	WO				
	I2		FI	DI	FO	EI	WO			
	I3			Idle	FI	DI	FO	EI	WO	
	I4					FI	DI	FO	EI	WO

(b) I1 source operand in memory

Figure 14.15 Example of Resource Hazard



		Clock cycle									
		1	2	3	4	5	6	7	8	9	10
ADD EAX, EBX		FI	DI	FO	EI	WO					
			FI	DI	Idle		FO	EI	WO		
	I3			FI			DI	FO	EI	WO	
	I4						FI	DI	FO	EI	WO

Figure 14.16 Example of Data Hazard

+ Types of Data Hazard

- Read after write (RAW), or true dependency
 - An instruction modifies a register or memory location
 - Succeeding instruction reads data in memory or register location
 - Hazard occurs if the read takes place before write operation is complete
- Write after read (WAR), or antidependency
 - An instruction reads a register or memory location
 - Succeeding instruction writes to the location
 - Hazard occurs if the write operation completes before the read operation takes place
- Write after write (WAW), or output dependency
 - Two instructions both write to the same location
 - Hazard occurs if the write operations take place in the reverse order of the intended sequence



Control Hazard

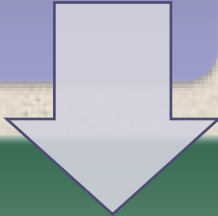


- Also known as a *branch hazard*
- Occurs when the pipeline makes the wrong decision on a branch prediction
- Brings instructions into the pipeline that must subsequently be discarded
- Dealing with Branches:
 - Multiple streams
 - Prefetch branch target
 - Loop buffer
 - Branch prediction
 - Delayed branch



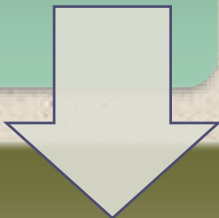
Multiple Streams

A simple pipeline suffers a penalty for a branch instruction because it must choose one of two instructions to fetch next and may make the wrong choice



A brute-force approach is to replicate the initial portions of the pipeline and allow the pipeline to fetch both instructions, making use of two streams

Drawbacks:

- 
- With multiple pipelines there are contention delays for access to the registers and to memory
 - Additional branch instructions may enter the pipeline before the original branch decision is resolved

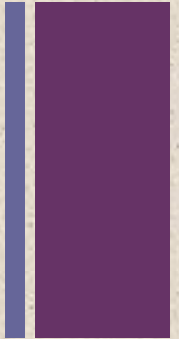
Prefetch Branch Target

- When a conditional branch is recognized, the target of the branch is prefetched, in addition to the instruction following the branch
- Target is then saved until the branch instruction is executed
- If the branch is taken, the target has already been prefetched
- IBM 360/91 uses this approach





Loop Buffer



- Small, very-high speed memory maintained by the instruction fetch stage of the pipeline and containing the n most recently fetched instructions, in sequence
- Benefits:
 - Instructions fetched in sequence will be available without the usual memory access time
 - If a branch occurs to a target just a few locations ahead of the address of the branch instruction, the target will already be in the buffer
 - This strategy is particularly well suited to dealing with loops
- Similar in principle to a cache dedicated to instructions
 - Differences:
 - The loop buffer only retains instructions in sequence
 - Is much smaller in size and hence lower in cost

Branch address

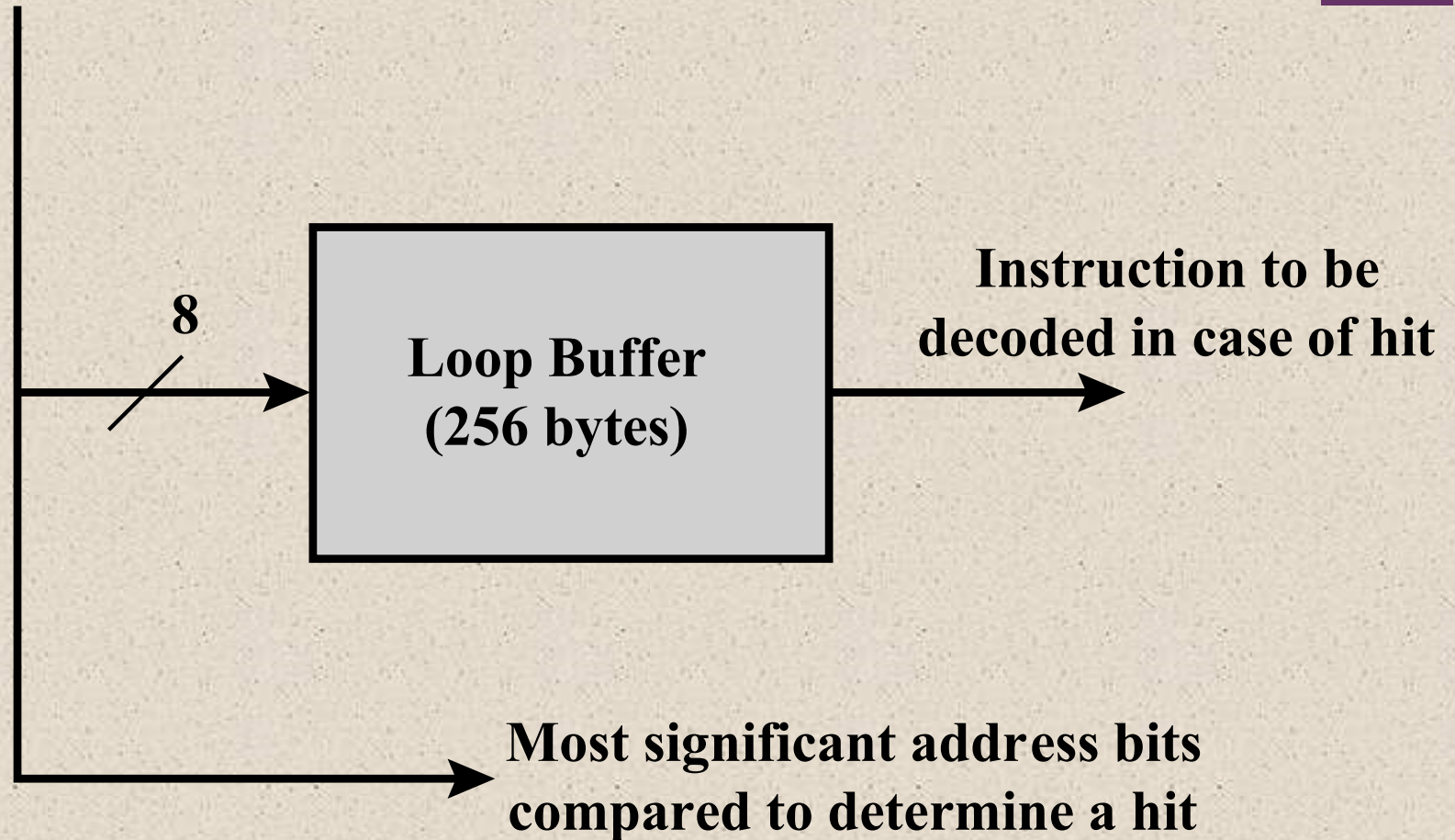


Figure 14.17 Loop Buffer



Branch Prediction



- Various techniques can be used to predict whether a branch will be taken:

1. Predict never taken

2. Predict always taken

3. Predict by opcode

- These approaches are static

- They do not depend on the execution history up to the time of the conditional branch instruction

1. Taken/not taken switch

2. Branch history table

- These approaches are dynamic

- They depend on the execution history

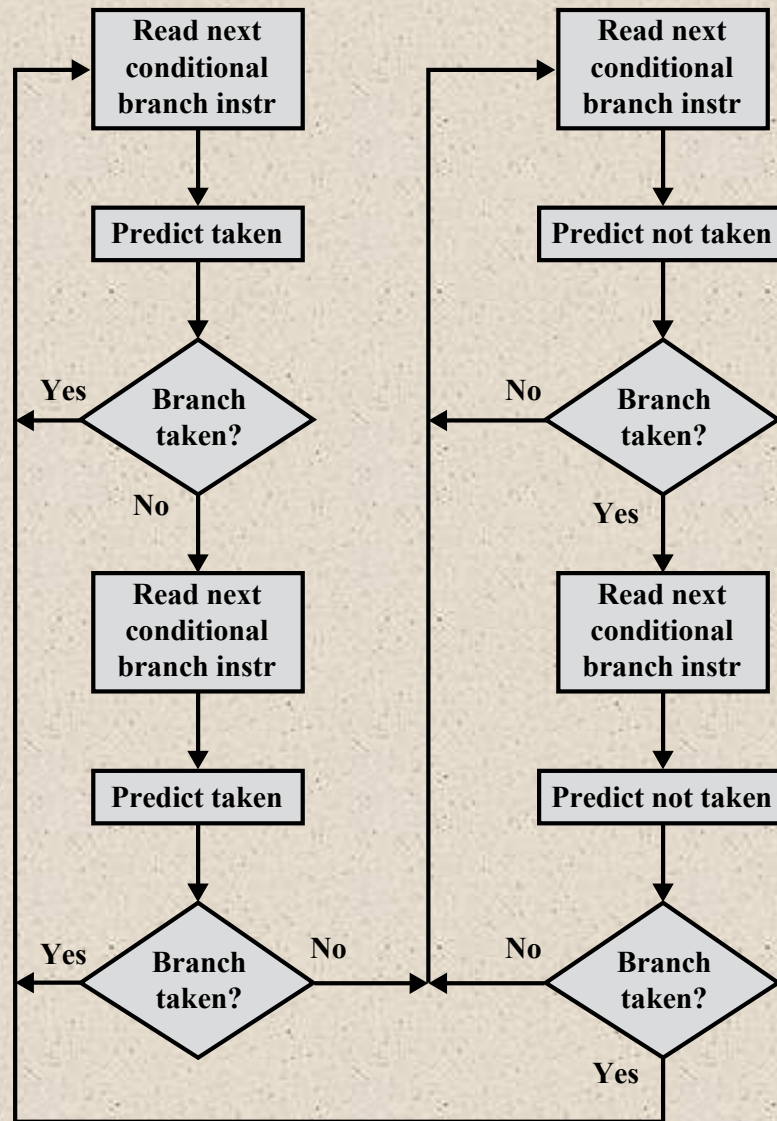


Figure 14.18 Branch Prediction Flow Chart

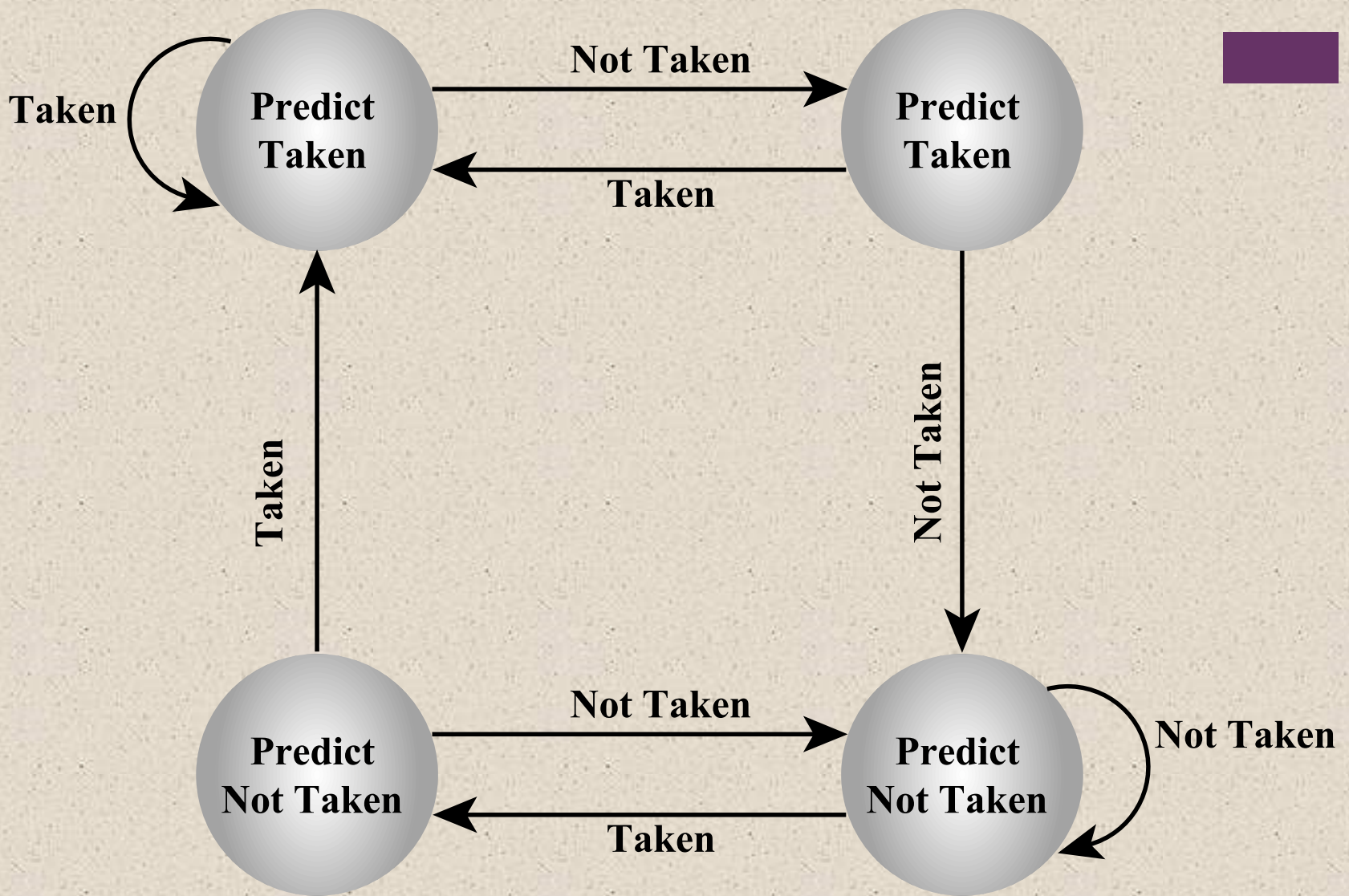


Figure 14.19 Branch Prediction State Diagram

+ Summary

Chapter 14

- Processor organization
- Register organization
 - User-visible registers
 - Control and status registers
- Instruction cycle
 - The indirect cycle
 - Data flow
- The x86 processor family
 - Register organization
 - Interrupt processing

Processor Structure and Function

- Instruction pipelining
 - Pipelining strategy
 - Pipeline performance
 - Pipeline hazards
 - Dealing with branches
 - Intel 80486 pipelining
- The Arm processor
 - Processor organization
 - Processor modes
 - Register organization
 - Interrupt processing